LAPORAN WORKSHOP DEVOPS

MINGGU KE-9

MATERI PEMBAHASAN:

Docker Swarm

oleh

Alvin Aura Puspa Santika

E41230289

Golongan A

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

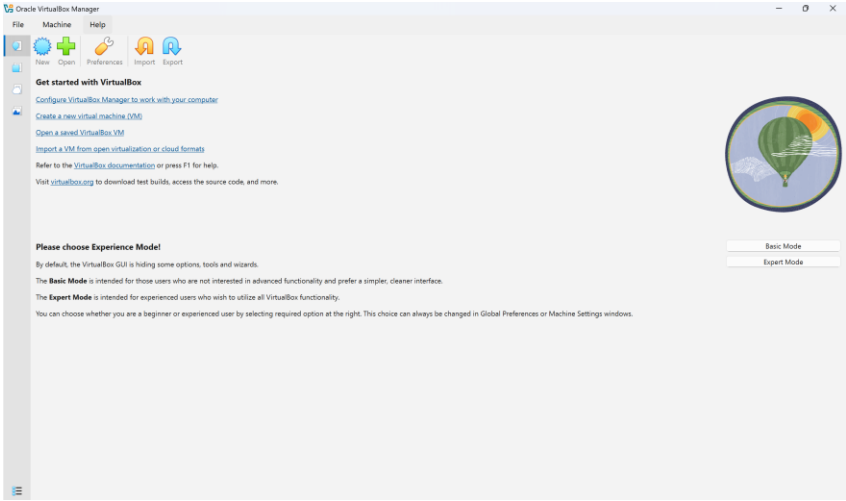
2026

BAB 1. HASIL DAN PEMBAHASAN

1.1 Persiapan Awal

1. *Install VistualBox*

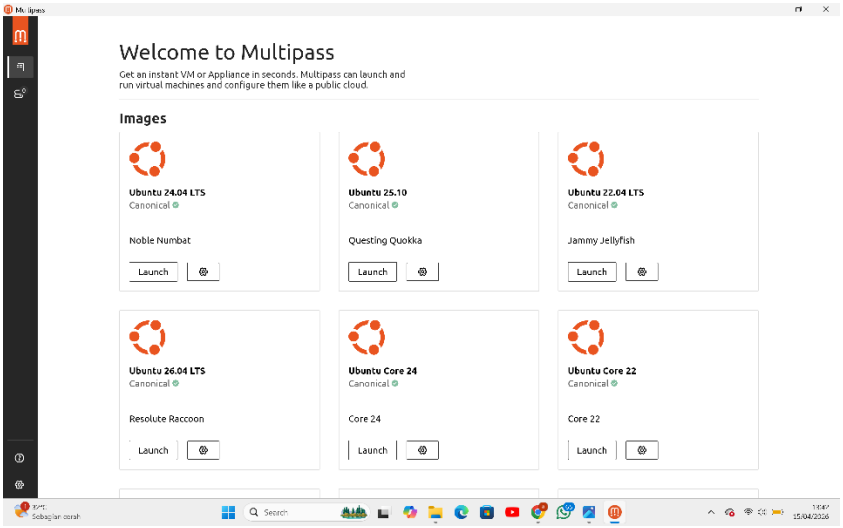
Langkah pertama yang dilakukan adalah installasi VirtualBox yang digunakan sebagai hypervisor untuk menjalankan mesin virtual. Hal yang saya lakukan yaitu mengunjungi situs resmi VirtualBox dan melakukan pengunduhan dari sana, lalu install sesuai system operasi.



Gambar 1. 1 Tampilan awal VirtualBox

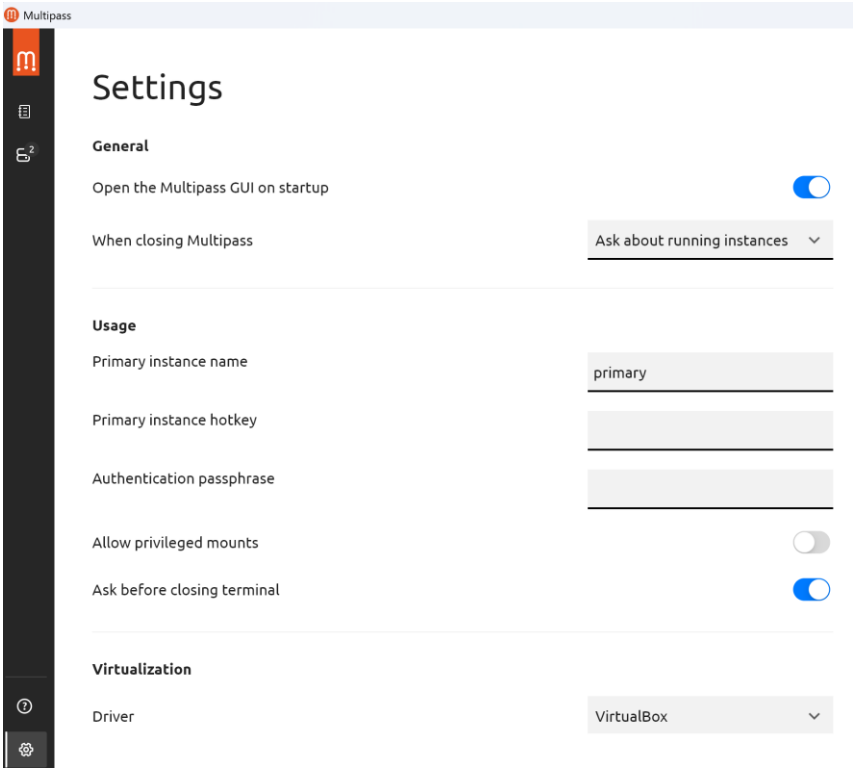
2. *Install Multipass*.

Langkah selanjutnya adalah melakukan installasi Multipass yang digunakan untuk membuat dan mengelola VM berbasis Ubuntu secara cepat.



Gambar 1. 2 Tampilan awal Multipass

3. Pengaturan *Multipass*

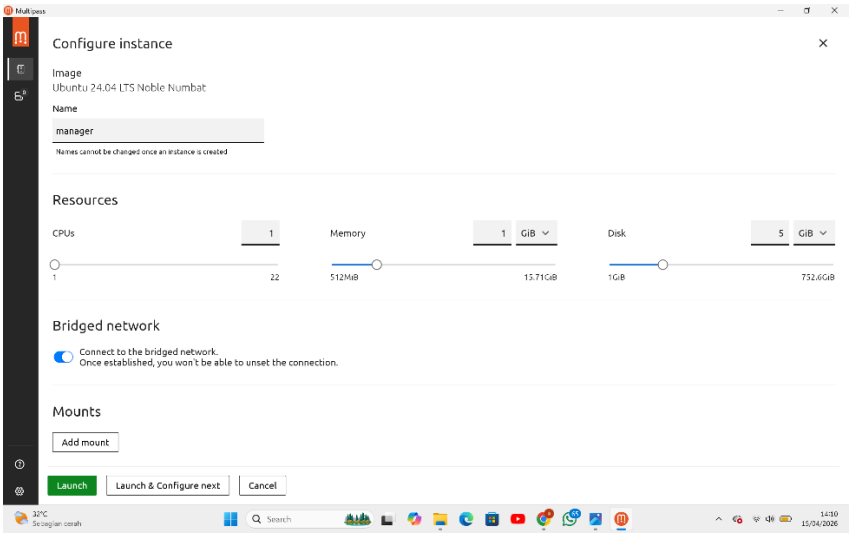


Gambar 1. 3 Pengaturan pada Multipass

1.2 Acara 17 (Konfigurasi Docker Swarm)

1. Membuat VM *manager dan worker*

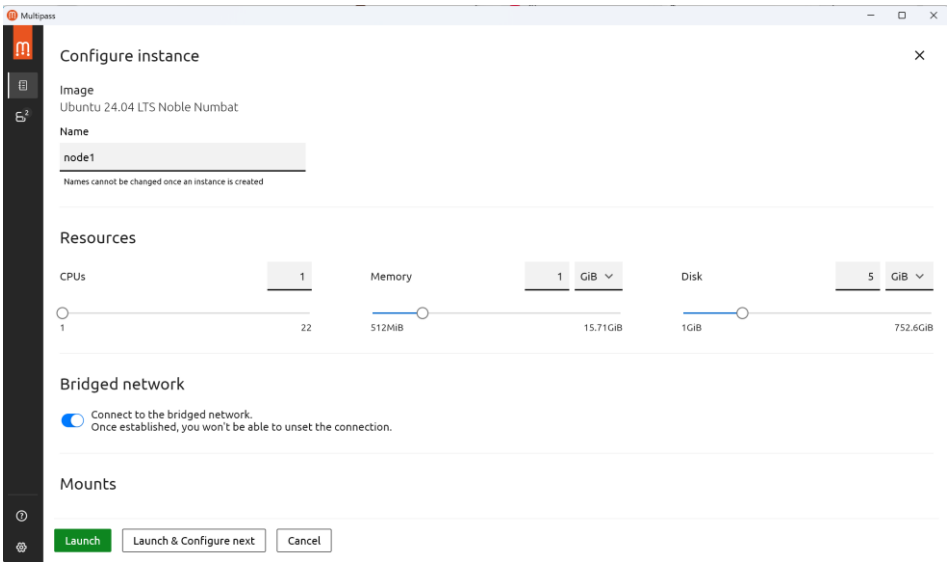
a. Membuat konfigurasi untuk manager seperti di gambar bawah ini.



Gambar 1. 4 Konfigurasi manager

Setelah itu, klik *launch*.

b. Membuat *worker* (saya beri nama Node1) dengan cara melakukan konfigurasi yang sama seperti pada *manager*, hanya saja namnya diisi dengan “Node 1”.

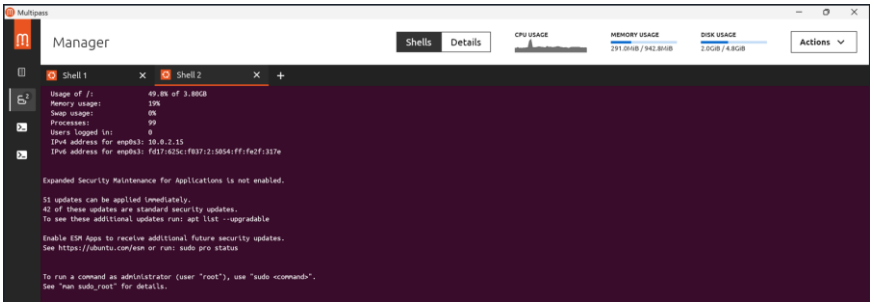


Gambar 1. 5 Konfigurasi worker

Setelah itu, klik *launch*.

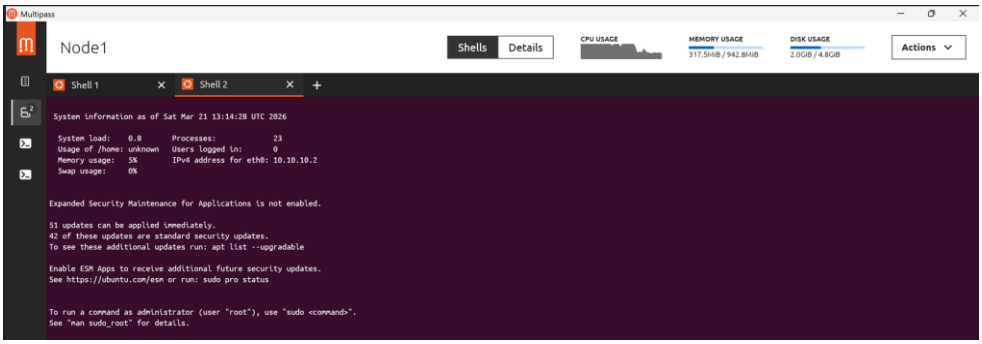
2. Masuk ke masing masing VM

a. Masuk ke *VM shell manager*



Gambar 1. 6 Masuk VM shell manager

b. Masuk ke VM *shell worker*



Gambar 1. 7 Masuk VM shell worker

3. Cek alamat IP masing-masing node, untuk mengetahui IP setiap VM yang akan digunakan sebagai Alamat komunikasi antar node dalam cluster.
   a. Cek Alamat IP *manager*

```
ubuntu@Manager:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 52:54:00:2f:31:7e brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
       valid_lft 85887sec preferred_lft 85887sec
    inet6 fd17:625c:f037:2:5054:ff:fe2f:317e/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86306sec preferred_lft 14306sec
    inet6 fe80::5054:ff:fe2f:317e/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 52:54:00:01:9d:8f brd ff:ff:ff:ff:ff:ff
    inet 10.10.182.75/22 metric 200 brd 10.10.183.255 scope global dynamic enp0s8
       valid_lft 89sec preferred_lft 89sec
    inet6 fe80::5054:ff:fe01:9d8f/64 scope link
       valid_lft forever preferred_lft forever
```

Gambar 1. 8 IP manager

   b. Cek Alamat IP *worker*

```
ubuntu@Node1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 52:54:00:33:6f:1c brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
       valid_lft 86223sec preferred_lft 86223sec
    inet6 fd17:625c:f037:2:5054:ff:fe33:6f1c/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86227sec preferred_lft 14227sec
    inet6 fe80::5054:ff:fe33:6f1c/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 52:54:00:7d:55:1c brd ff:ff:ff:ff:ff:ff
    inet 10.10.182.94/22 metric 200 brd 10.10.183.255 scope global dynamic enp0s8
       valid_lft 64sec preferred_lft 64sec
    inet6 fe80::5054:ff:fe7d:551c/64 scope link
       valid_lft forever preferred_lft forever
```

Gambar 1. 9 IP worker

4. *Install docker* pada kedua *node*
   a. *Install docker* pada *shell manager*

```
ubuntu@Manager:~$ sudo apt install docker.io -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools cgroupfs-mount | cgroup-lite debootstrap docker-buildx docker-compose-v2
  docker-doc rinse zfs-fuse | zfsutils
The following NEW packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base docker.io pigz runc ubuntu-fan
0 upgraded, 8 newly installed, 0 to remove and 46 not upgraded.
Need to get 73.8 MB of archives.
After this operation, 279 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/main amd64 bridge-utils amd64 1.7.1-1ubuntu2 [33.9 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 runc amd64 1.3.4-0ubuntu1~24.04.1 [957
```

Gambar 1. 10 Install docker pada manager

```
ubuntu@Manager:~$ sudo systemctl enable docker
ubuntu@Manager:~$ sudo systemctl start docker
ubuntu@Manager:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.1
```

Gambar 1. 11 Docker berhasil terinstall pada node manager

b. *Install docker* pada *shell Node1*

Memberikan perintah yang sama seperti pada *shell manager* untuk menginstall *docker*.

```
ubuntu@Node1:~$ sudo systemctl enable docker
ubuntu@Node1:~$ sudo systemctl start docker
ubuntu@Node1:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.1
```

Gambar 1. 12 Docker berhasil terinstall pada node 1

5. Inisialisasi *swarm* pada *manager*, untuk menjadikan node manager sebagai pusat cluster yang mengatur semua *node* dan *service* dalam *Docker Swarm*.

```
ubuntu@Manager:~$ sudo docker swarm init --advertise-addr 192.168.1.27
Swarm initialized: current node (arp5vnokw520d65qcfsc6l3fs) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-2ifm7vhjz6ualb98gogpo3bckkaqq3tuwsbtmgovhnn2hcwtmu-0szjebqlgwxc
akrb57c6uirep 192.168.1.27:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

Gambar 1. 13 Inisialiasi swarm pada manager

6. Join *worker* ke *swarm*, untuk menghubungkan *node worker* ke *manager* agar menjadi bagian dari cluster dan dapat menjalankan task.

   Caranya dengan melakukan copy dan paste command yang telah muncul pada saat inisialisasi swarm.

```
ubuntu@Node1:~$ sudo docker swarm join --token SWMTKN-1-2ifm7vhjz6ualb98gogpo3bckkaqq3tuwsbtmgovhnn2hc
wtmu-0szjebqlgwxcakrb57c6uirep 192.168.1.27:2377
This node joined a swarm as a worker.
```

Gambar 1. 14 Join worker ke swarm

7. Verifikasi *node*, untuk memastikan semua *node* sudah terhubung dan aktif dalam cluster.

```
ubuntu@Manager:~$ sudo docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
56evkumbrk30tzbc4t7ert92j *   Manager    Ready     Active         Leader           29.1.3
53r914s57hmlghjgc0zasnyiu     Node1      Ready     Active                          29.1.3
```

Gambar 1. 15 Verifikasi node

1.3 Acara 18 (Mengelola Service)

1. Membuat *service* Nginx yang akan dijalankan sebagai aplikasi *web* dalam *cluster*.

```
ubuntu@Manager:~$ sudo docker service create --name web -p 80:80 nginx
0e900gvvrgsvc6xiuaxshs4zy
overall progress: 1 out of 1 tasks
1/1: running
verify: Service 0e900gvvrgsvc6xiuaxshs4zy converged
```

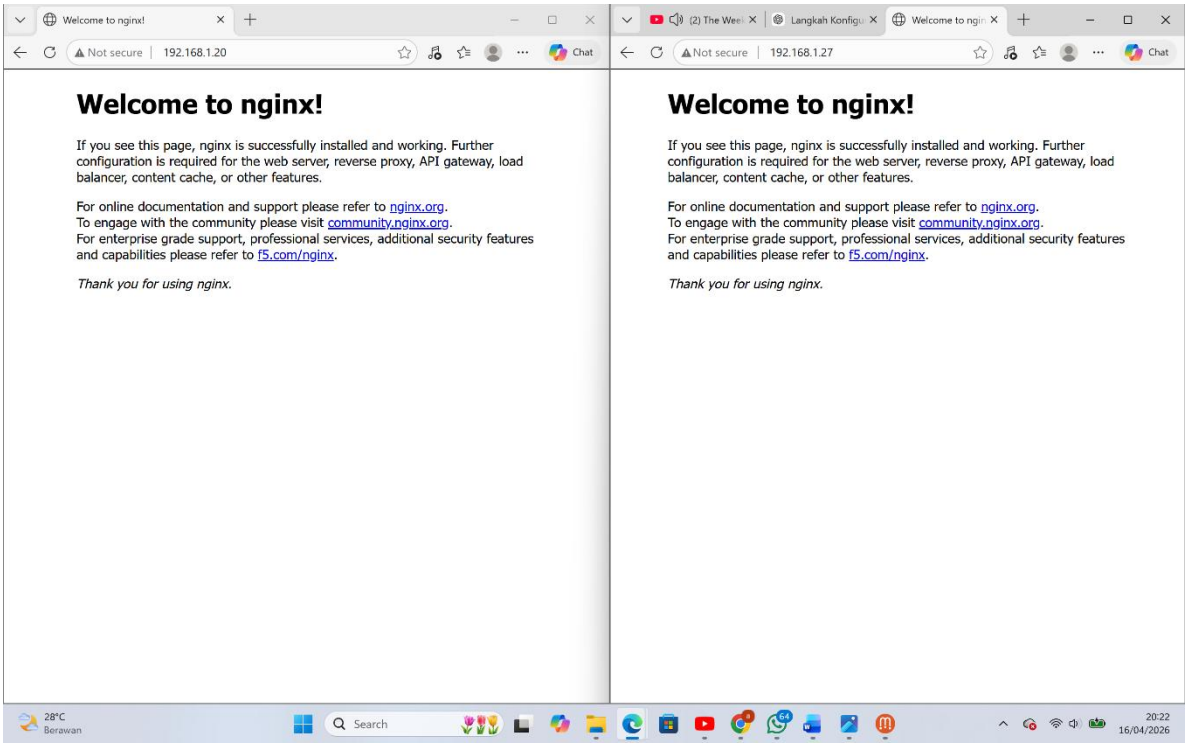
Gambar 1. 16 Membuat service nginx

Lalu cek *service* nginx

```
ubuntu@Manager:~$ sudo docker service ls
ID             NAME      MODE         REPLICAS   IMAGE          PORTS
0e900gvvrgsv   web       replicated   1/1        nginx:latest   *:80->80/tcp
```

Gambar 1. 17 Cek service nginx

Test di *Browser* untuk memastikan bahwa service nginx berhasil dibuat



2. Melakukan *update service* dari awalnya menggunakan *image* nginx menjadi image lain, contohnya di sini digunakan image igdw/first-repo yang ada di *Docker Hub*.

```
ubuntu@Manager:~$ sudo docker service update --image igdw/first-repo web
web
overall progress: 1 out of 1 tasks
1/1: running
verify: Service web converged
```

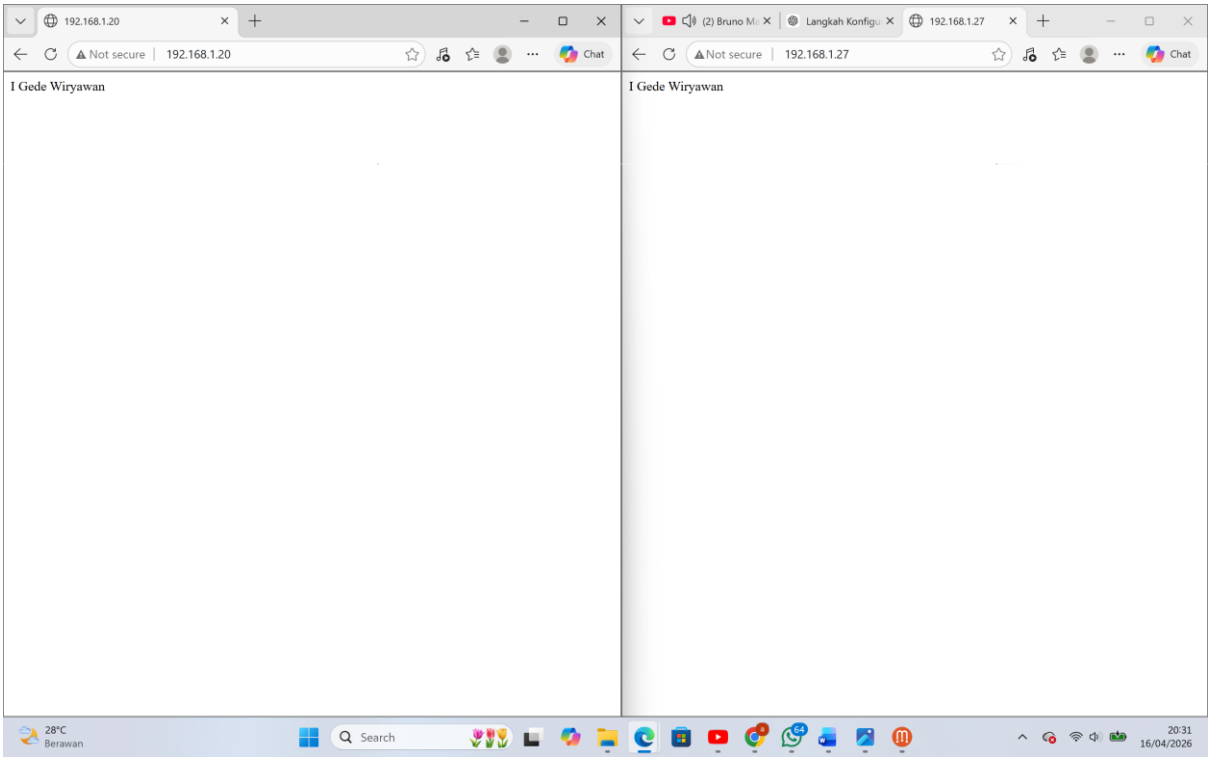
Gambar 1. 18 Update service

Cek *service*

```
ubuntu@Manager:~$ sudo docker service ps web
ID             NAME        IMAGE                    NODE      DESIRED STATE   CURRENT STATE           ERROR     PORTS
3z6uevxjt0gz   web.1       igdw/first-repo:latest   Manager   Running         Running 3 minutes ago
tw6eqx0cvjg5    \_ web.1   nginx:latest             Manager   Shutdown        Shutdown 5 minutes ago
utv94gjgu8qs    \_ web.1   nginx:latest             Node1     Shutdown        Shutdown 8 minutes ago
udjuxmyy1uf1    \_ web.1   nginx:latest             Manager   Shutdown        Shutdown 8 minutes ago
```

Gambar 1. 19 Cek service

Test di *Browser* untuk memastikan bahwa *service* nginx berhasil di update



Gambar 1. 20 Halaman web setelah update service nginx

3. *Inspect service* di *manager*

```
ubuntu@Manager:~$ sudo docker service inspect web --pretty

ID:             0e900gvvrgsvc6xiuaxshs4zy
Name:           web
Service Mode:   Replicated
 Replicas:      1
UpdateStatus:
 State:         completed
 Started:       7 minutes ago
 Completed:     5 minutes ago
 Message:       update completed
Placement:
UpdateConfig:
 Parallelism:   1
 On failure:    pause
 Monitoring Period: 5s
 Max failure ratio: 0
 Update order:      stop-first
RollbackConfig:
 Parallelism:   1
 On failure:    pause
 Monitoring Period: 5s
 Max failure ratio: 0
 Rollback order:    stop-first
ContainerSpec:
 Image:         igdw/first-repo:latest
 Init:          false
Resources:
Endpoint Mode:  vip
Ports:
 PublishedPort = 80
  Protocol = tcp
  TargetPort = 80
  PublishMode = ingress
```

Gambar 1. 21 Inspect service manager

Perintah ini digunakan untuk melihat detail service seperti nama service, jumlah replica, port, dan node yang menjalankan.

4. *Inspect service* di *worker*

```
ubuntu@Node1:~$ sudo docker service inspect web --pretty
Error response from daemon: This node is not a swarm manager. Worker nodes can't be used to view or mo
dify cluster state. Please run this command on a manager node or promote the current node to a manager
```

Gambar 1. 22 Inspect service worker

Outupnya akan menunjukkan error karena worker tidak memiliki hak control swarm, maka hanya manager yang bisa inspect service.

5. Cek *service running* untuk menampilkan *container* yang sedang berjalan dan memfilter hanya yang status *running*.

```
ubuntu@Manager:~$ sudo docker service ps web | grep Running
3z6uevxjt0gz   web.1       igdw/first-repo:latest   Manager   Running        Running 11 minutes ago
```

Gambar 1. 23 Cek service running

6. *Worker* keluar dari *swarm* dan tidak lagi menerima *task*.

```
ubuntu@Node1:~$ sudo docker swarm leave
Node left the swarm.
```

Gambar 1. 24 Worker keluar dari swarm

7. *Manager* keluar dari *swarm* dengan paksa karena dia adalah pusat *cluster*.

```
ubuntu@Manager:~$ sudo docker swarm leave --force
Node left the swarm.
```

Gambar 1. 25 Manager keluar dari swarm